YAFL
Yet Another f. . . Language

Fabian Becker

January 21, 2026

Abstract

This report details the design and implementation of YAFL, a statically and strongly typed procedural programming language submitted for the Compiler Construction study project. Designed to explicitly deviate from C-style syntax and semantics, YAFL features a unique pipe-based identifier system, base-N numeric literals, and lazy iterator abstractions. The language compiles to custom bytecode for an extended architecture of the provided VM3 virtual machine, which was modified to support custom opcodes for iterators, slicing, and File I/O functions.

1 General Overview

YAFL stands for "Yet Another F... Language". The variable "F" reflects the programmer's current sentiment towards the language: *Fabulous* when it compiles, *Fun* when writing algorithms, or *Frustrating* when debugging.

1.1 Design Philosophy

The core design objective of YAFL is to merge the readability of modern scripting languages with the structural discipline of traditional system languages.

- **Expressiveness:** While function signatures resemble Go, the language adopts high-level concepts like generalized iterators and range-based loops (inspired by Python) to reduce boilerplate code.
- **Structure & Safety:** YAFL retains explicit structure using brace delimiters {} and semicolons ;. Crucially, it enforces a **static and strong type system** to identify errors at compile time rather than runtime, prioritizing reliability over flexibility.

1.2 Compiler Architecture

The YAFL compiler is implemented in C and utilizes a pipelined architecture to transform source code into executable bytecode. The compilation process consists of the following stages:

1. **Lexical Analysis:** Implemented using Flex, this stage scans the input source code and converts character streams into a sequence of tokens.
2. **Parsing:** Implemented using Bison, this stage consumes the token stream to construct the Abstract Syntax Tree (AST), representing the hierarchical structure of the program.
3. **Optimization:** Additional pre-processing passes traverse the AST to perform Constant Folding and Dead Code Elimination. This simplifies the tree structure before it is processed further.
4. **Code Generation & Semantic Analysis:** The AST is traversed to lower the program into the custom bytecode instruction set for the VM3 virtual machine. During this traversal, semantic analysis is performed on-the-fly. This includes symbol resolution, strict type checking, and argument validation, ensuring that only valid and type-safe programs result in executable bytecode.

2 Syntax and Conventions

YAFL introduces several unique syntactic elements that distinguish it from standard C-family languages.

Feature	Syntax / Operator	Description & Examples
Identifiers	`|variable name|`	Enclosed in pipes to separate them from keywords. This allows for **expressive names with spaces**. **Examples:** `|user input|`, `|my counter|`
Functions	`fn ... -> type`	Explicit return type arrow. **Examples:** 1. `fn add(int:|a|, int:|b|) -> int {...}` 2. `fn start() -> none {...}`
Entry Point	`start()`	The program execution begins at the function named `start`, chosen to clearly denote the entry point (analogous to `main` in C).
Comments	`~~`, `{- ... -}`	Line comments start with `~~`. Block comments use curly braces.

Table 1: YAFL Syntax Overview

2.1 Numeric and String Literals

Moreover the language supports a highly flexible number format. Beyond standard decimals, numbers can be prefixed with a base (2-16) using the hash syntax `base#number`. If no base was provided (`#number`), the literal will be interpreted as a hexadecimal number.

- **Base-N Floats:** This base logic extends to floating point numbers. Both the integral and fractional parts are interpreted in the specified base (e.g., `16#F.8` equals 15.5 decimal).
- **Formatting:** Underscores are ignored to allow digit grouping (e.g., `10#1_000`).

String literals are delimited by double chevrons (`»...«`), avoiding standard quoting issues. The caret (`^`) serves as the escape character, supporting hex insertion (e.g., `^xAA`) and standard escapes like `^n`.

2.2 Assignments and Operators

The assignment operator is `<-`, emphasizing data flow. This allows the equals sign `=` to be repurposed for comparison, a significant deviation from `C`.

- **Comparison:** Equality is checked using `=` (not `==`). Inequality is denoted by `~=` (resembling `MATLAB`), replacing the traditional `!=`.
- **Polymorphic Operators:** Some arithmetic operators are overloaded for types other than numbers, specifically:
  - **Concatenation (+):** Joins two strings or merges two arrays.
  - **Repetition (*):** Repeating a string or array n times (e.g., `»ab« * 3` results in `»ababab«`).
- **Logic & Bitwise:** The keywords `and`, `or` and `not` are used for logical and bitwise operations, replacing `&&`, `||` and `!`.
- **Increment/Decrement:** The operators `++` and `--` are postfix-only. Unlike `C`, they execute the operation and **return the updated value** immediately.

3 Language Features

3.1 Type System

YAFL is **statically and strongly typed**, ensuring strict type safety at compile time with zero-initialization semantics. Implicit casting is not permitted; conversions must be explicit (e.g., `to_float`). The type system supports:

- **Primitives:** `int`, `float`, `bool`, and `str`. Primitives are passed by reference.
- **Collections:** `arr` (typed arrays) and `range'int` (lazy sequences).
- **Special:** `none` for empty returns.

Composite types (e.g., `arr'arr'str`) are supported. Uninitialized variables default to zero values (0, 0.0, `No`, or »«).

3.2 Control Structures

Conditionals: Standard `if-elif-else` structures are converted to nested `if` statements.

Loops & Iterators: The `while` loop operates on boolean conditions. The `for` loop uses iterators (syntax: `for <type> |v| in iterator`).

- **Lazy Evaluation:** The `range()` function is implemented as a lazy iterator. Instead of creating a full array on the virtual machine stack, the compiler emits the `MKRANGE` opcode, creating a lightweight state object `["__RANGE__", current, stop, step]` on the stack.
- **Generic Iteration:** Arrays and strings are wrapped by the `MKITER` opcode into a uniform container (`[iterable, current, len]`). The `ITER_NEXT` opcode handles traversal transparently, by updating the iterator and pushing the next value or an exit signal.
- **Flow Control:** The operators `next` and `stop` (equivalent to continue/break) utilize a compile-time "loop stack". This stack tracks jump targets, allowing the compiler to back-patch exit addresses once the loop body is fully generated.

Match Statement: The `match` statement can replace complex `if-else` chains. Since `YAFL` allows matching on strings, a simple jump table is insufficient.

- **Implementation:** The compiler generates a hashmap mapping case literals to bytecode addresses.
- **Execution:** The custom `SWITCH_LOOKUP` opcode performs a hash lookup at runtime to determine the jump target, ensuring $O(1)$ performance even for string keys.

3.3 Function Architecture

`YAFL` supports advanced function features requiring a custom symbol table and multi-pass compilation.

- **Unified Symbol Table:** Built-in functions are pre-registered in the same custom symbol table as user-defined functions. This simplifies the parser, as built-ins like `args()` (which returns user arguments excluding the program name) are treated exactly like standard calls.
- **Execution Strategy (CALL_PC):** The standard `VM3` virtual machine resolves function calls by name. This is insufficient for `YAFL's` overloading support, where multiple functions can share the same name. To solve this, the custom `CALL_PC` opcode was implemented. It dispatches calls to a specific bytecode address (Program Counter), ensuring the correct overloaded signature is targeted.
- **Overloading:** Functions are identified by a signature combining the function name argument types and their order, allowing multiple definitions (e.g., specific `to_int` implementations for different types).

- **Default Arguments:** Trailing arguments can have default values. The compiler enforces that all subsequent parameters also possess defaults.
- **Forward References (Two-Pass Generation):** To allow calling functions before their definition, the compiler uses a two-pass approach.
    1. **Registration:** In the first pass all function signatures are registered in the symbol table.
    2. **Body Generation:** During the second pass function bodies are compiled. If a call references a not-yet-compiled function, the call site is added to a "fixup list."
    3. **Patching:** Once the target function is compiled, its start address is back-patched into all pending call sites in the fixup list.

4 Optimizations

To ensure efficient bytecode execution, the compiler performs a dedicated optimization pass on the AST prior to code generation.

- **Constant Folding:** Arithmetic expressions involving literals are evaluated at compile time. For example, sub-expressions like `3 * 4 + 2` are reduced to `14` in the AST, decreasing runtime instruction count.
- **Dead Code Elimination:**
    - **Unreachable Statements:** Code following control flow interruptions (e.g., `return`, `next`, `stop`) is detected and pruned.
    - **Branch Pruning:** `if` and `match` structures with constant conditions are simplified. If a condition is known to be false at compile time, the entire branch is discarded; if true, the conditional wrapper is removed, and only the body is retained.

5 Toolchain and Usage

The `YAFL` ecosystem mimics a standard two-stage compilation process, distinguishing between compilation and execution environments.

- **Compiler (`yaflc`):** Transforms source files (`.yafl`) into bytecode (`.yaflb`). It supports the `-v` flag to export the AST as Graphviz `.dot` and `.svg` files, allowing for visual verification of the parsing and optimization stages.
- **Bytecode Executor (`yafl`):** A modified version of `VM3` that executes the bytecode. It accepts command-line arguments, which are accessible to the user program via the `args()` builtin function.

6 Demonstration Programs

The capabilities of `YAFL` are demonstrated through four included programs:

1. **Game of Life**: Implements Conway's Game of Life to showcase the language's handling of multi-dimensional arrays and nested loops. It is able to read patterns from standard `.rle` files to create an animated visualization in the terminal.
2. **Run Length Decoder**: A recursive descent parser for decoding run-length encoded strings (e.g., `43[ab]`). This program demonstrates recursion, string manipulation, and conditional logic.
3. **Black Jack**: A fully interactive card game simulation. It highlights the use of the `rng` builtin for game logic, `input_str` for user interaction, and extensive control flow using `match` and `if-else` structures.

4. **Language Demo**: A comprehensive showcase of specific language features including function overloading, slicing, and pattern matching.

A Project Structure

The submitted project is structured in the following way:

- **demo:** Contains the demonstration programs
- **doc:** The LaTeX code for this report
- **src:** Compiler source code
- **test:** Various small programs written during implementation
- **vsc-syntax:** A very simple syntax highlighter extension for `Visual Studio Code`
- **vm3:** The code for the modified version of `VM3`

B List of Builtin Functions

Signature	Description
`args() -> arr'str`	Returns the command-line arguments passed to the program.
`contains(str:|haystack|, str:|needle|) -> int`	Returns the starting index of `needle` in `haystack`, or -1 if not found.
`copy(<arr|str>:|val|) -> <type>`	Creates a deep copy of an array or string.
`exit() -> none`	Immediately terminates the program.
`input_int(str:|prompt|) -> int`	Prints a prompt and reads an integer from stdin. For invalid integers 0 gets returned.
`input_str(str:|prompt|) -> str`	Prints a prompt and reads a line from stdin.
`len(<arr|str>:|val|) -> int`	Returns the number of elements in an array or characters in a string.
`print(any:|val|) -> none`	Prints the string representation of a value to stdout.
`println(any:|val|) -> none`	Prints the string representation of a value followed by a newline.
`range(int:|start|, int:|stop|, int:|step| <- 1) -> range`	Creates a range iterator from `start` (inclusive) to `stop` (exclusive) with step `step`.
`range(int:|stop|) -> range`	Creates a range iterator from 0 (inclusive) to `stop` (exclusive) with step 1.
`read(str:|path|) -> arr'str`	Reads a text file into an array of strings (lines).
`rng(int:|min|, int:|max|) -> int`	Generates a random integer between `min` and `max` (inclusive).
`rng(int:|max|) -> int`	Generates a random integer between 0 and `max` (inclusive).
`sleep(int:|ms|) -> none`	Pauses program execution for `ms` milliseconds.

Signature	Description
`slice(<arr|str>:|v|, int:|s| <- 0, int:|e| <- len(|v|)) -> <type>`	Returns a slice from index `s` (inclusive) to `e` (exclusive).
`to_bool(<int|float|str>:|v|) -> bool`	Converts or parses a value to a boolean.
`to_float(<int|bool|str>:|v|) -> float`	Converts or parses a value to a float.
`to_int(<float|bool|str>:|v|) -> int`	Converts or parses a value to an integer.
`to_str(<int|float|bool>:|v|) -> str`	Converts a value to its string representation.
`write(str:|s|, str:|txt|, bool:|app| <- No) -> bool`	Writes or appends text to a file. Returns success status.

C VM3 Modifications

To support YAFL's features, the standard VM3 architecture was extended with custom opcodes and native functions. Besides the already mentioned ones, these are the additionally created opcodes.

OPCODE/NATIVE Identifier	Description
ARGS	Pushes commandline argument array to stack
CONTAINS	Returns the index of the needle (str) in the haystack (str), -1 if not found
READ_FILE	Reads a file by returning a string array of lines (newline gets stripped)
SLEEPMS	Sleeps for the provided number of milliseconds ($\leq$10s) by using `nanosleep()`
SLICE	Slices a string, negative indexing (start from end) allowed, returns a deep copy
SWAP	Swaps the two topmost elements on the stack
WRITE_FILE	Writes or appends string to a file (without newline), returns success a boolean

Other modifications of the source code include:

- Renamed `stack_t` to `vmstack_t` due to name collision with stack.h required by signal.h.
- Rewrote type casting to include parsing of YAFLs custom number format from string. Added additional functionality for some types.
- Changed `exec_t` and `exec_new` to accept commandline arguments.
- `val_mul` now supports multiplication of arrays and numbers.
- `arr_copy` now creates deep copy even for nested types.
- `str_add_buf` uses exponential growth in order to trigger less reallocation when concatenating multiple strings.